# AWS VPC: Public and Private EC2 Setup

| | | | |
|---|---|---|---|
| **Name** | Mithun U | **Employee ID** | 133902 |

## 1. Create the VPC

Created a VPC named **mithun_vpc** with the IPv4 CIDR block **16.0.0.0/16**. The default tenancy setting was retained.

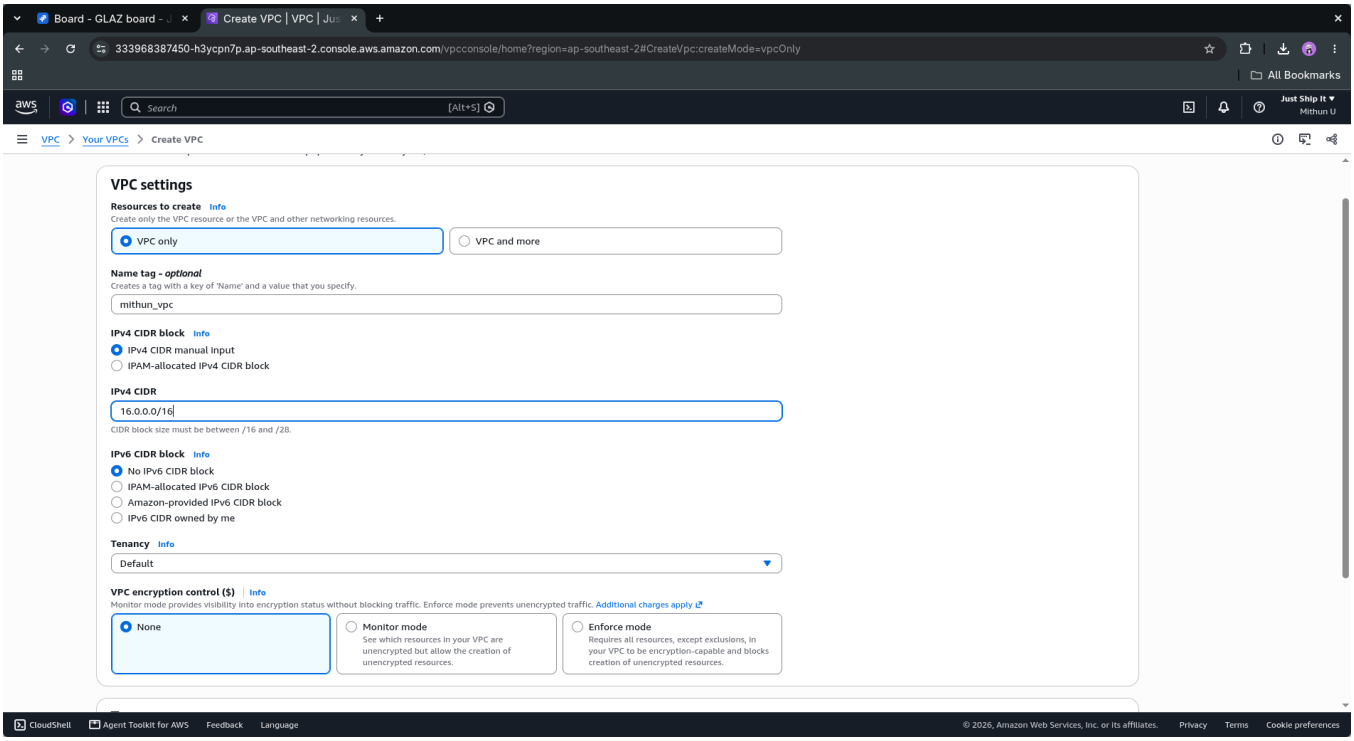



Figure 1. VPC creation settings.

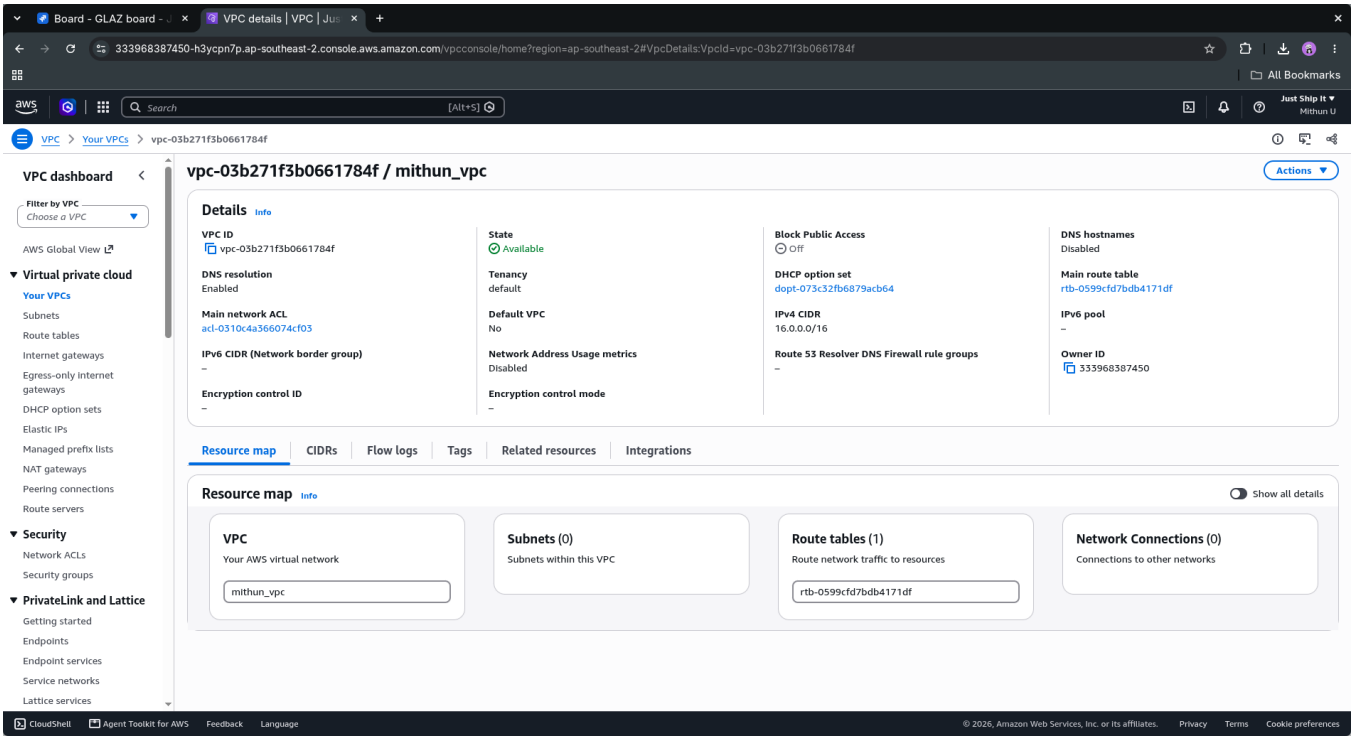



Figure 2. VPC created.

## 2. Create public and private subnets

Created two subnets inside mithun_vpc, using separate /24 CIDR blocks and Availability Zones.

| Subnet | Name | IPv4 CIDR | Availability Zone |
|---|---|---|---|
| Public | mithun_public_subnet | 16.0.10.0/24 | ap-southeast-2a |
| Private | mithun_private_subnet | 16.0.20.0/24 | ap-southeast-2b |

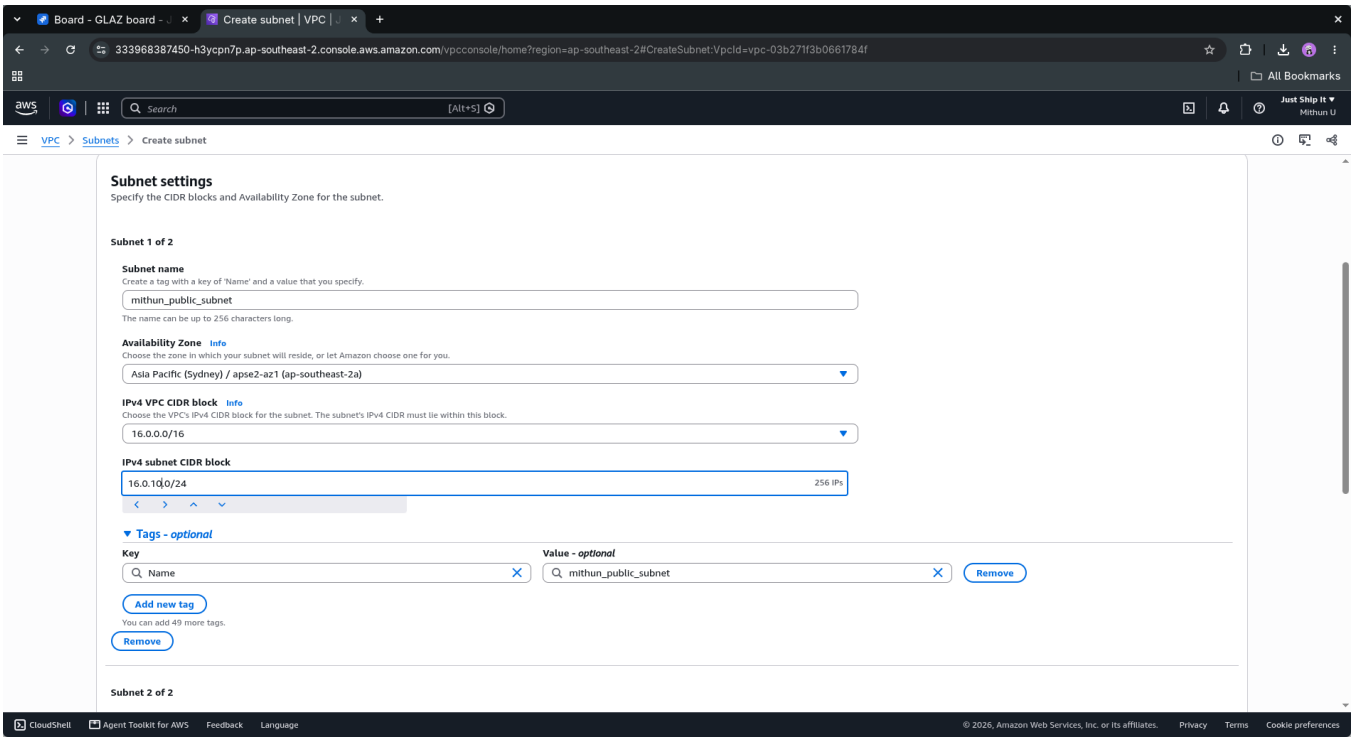



Figure 3. Public and private subnet configuration.

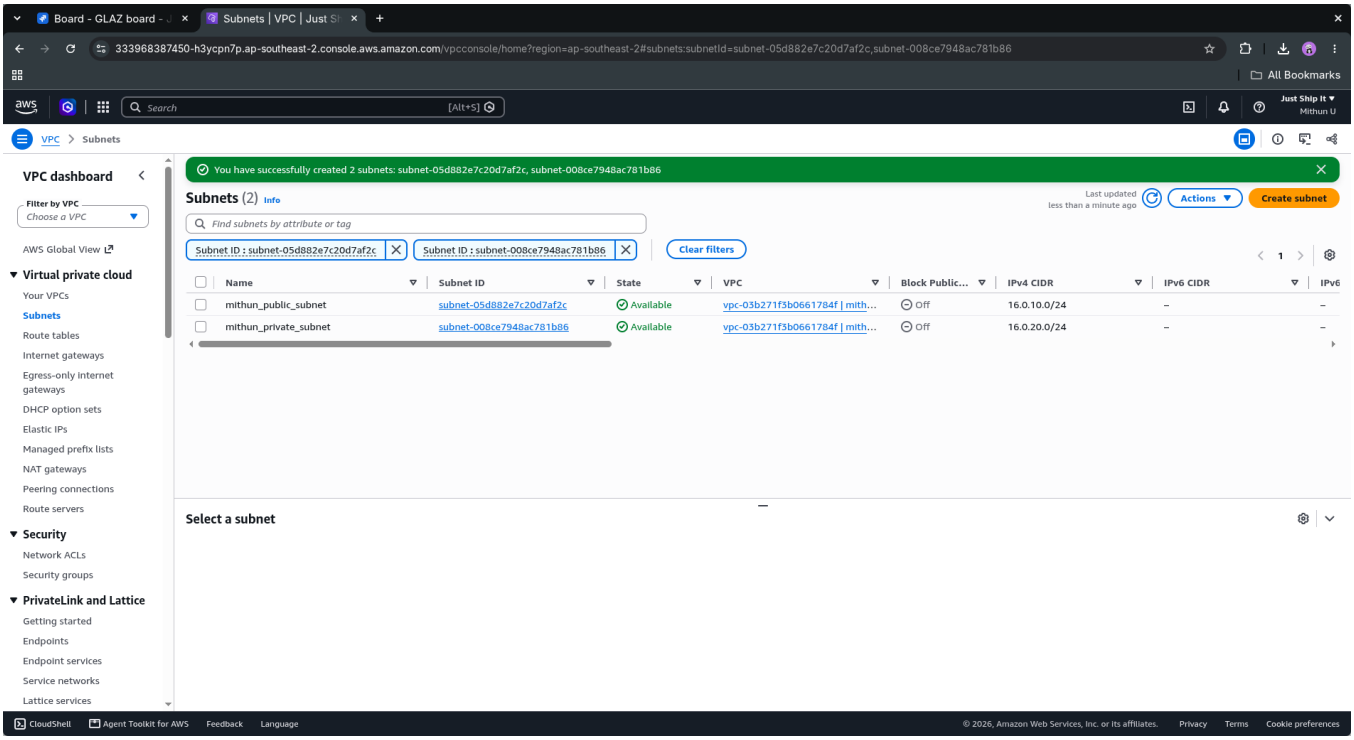



Figure 4. Subnets created.

# 3. Internet Gateway and public route table

Created an Internet Gateway named **mithun_igw** and attached it to mithun_vpc. Configured a public route table and associated it with mithun_public_subnet.

```
Destination: 0.0.0.0/0 → Internet Gateway (mithun_igw)
Destination: 16.0.0.0/16 → local
```

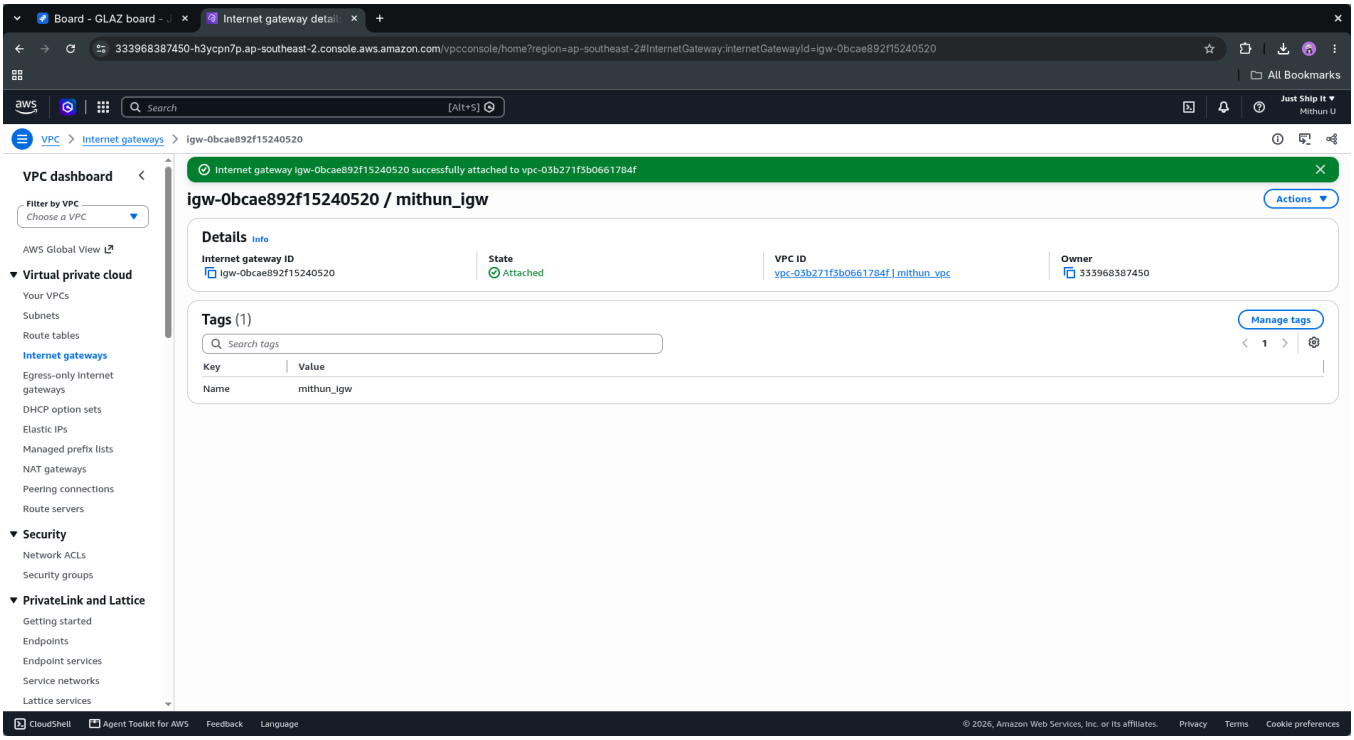



Figure 5. Internet Gateway attached to the VPC.

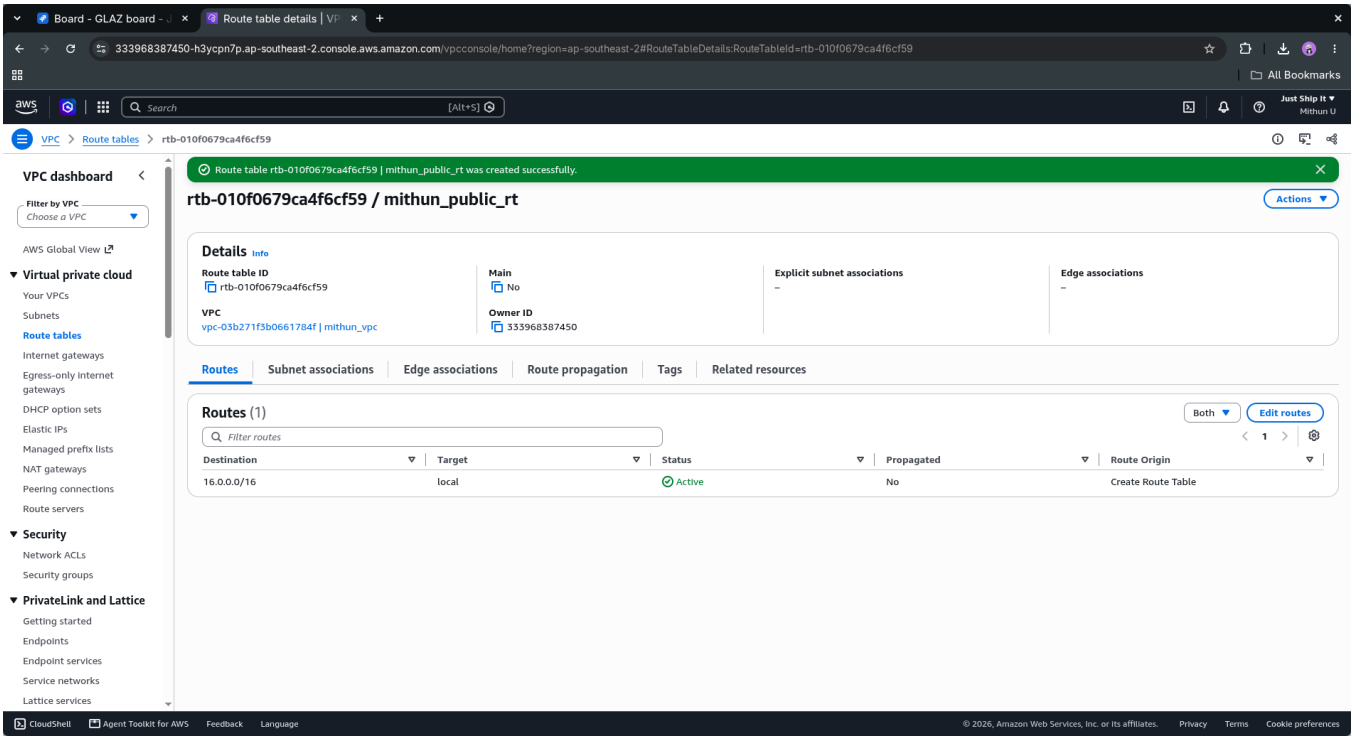



Figure 6. Public route table routes.

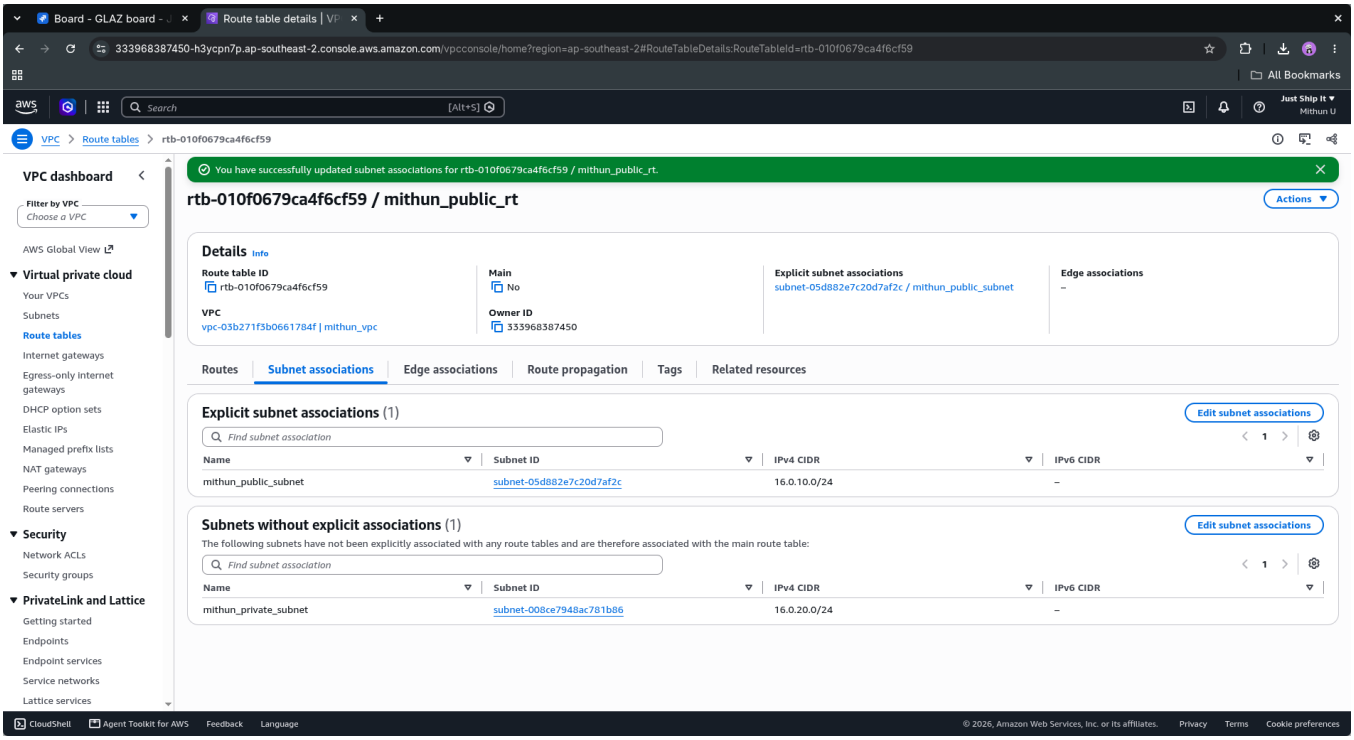



Figure 7. Public subnet association.

# 4. NAT Gateway and private route table

Created a public NAT Gateway named **mithun_nat_gateway** in the public subnet and allocated an Elastic IP address. Configured a private route table with the NAT Gateway as the default route, then associated it with mithun_private_subnet.

```
Destination: 0.0.0.0/0 → NAT Gateway (mithun_nat_gateway)
Destination: 16.0.0.0/16 → local
```

This allows instances in the private subnet to initiate outbound internet connections without assigning them public IPv4 addresses.

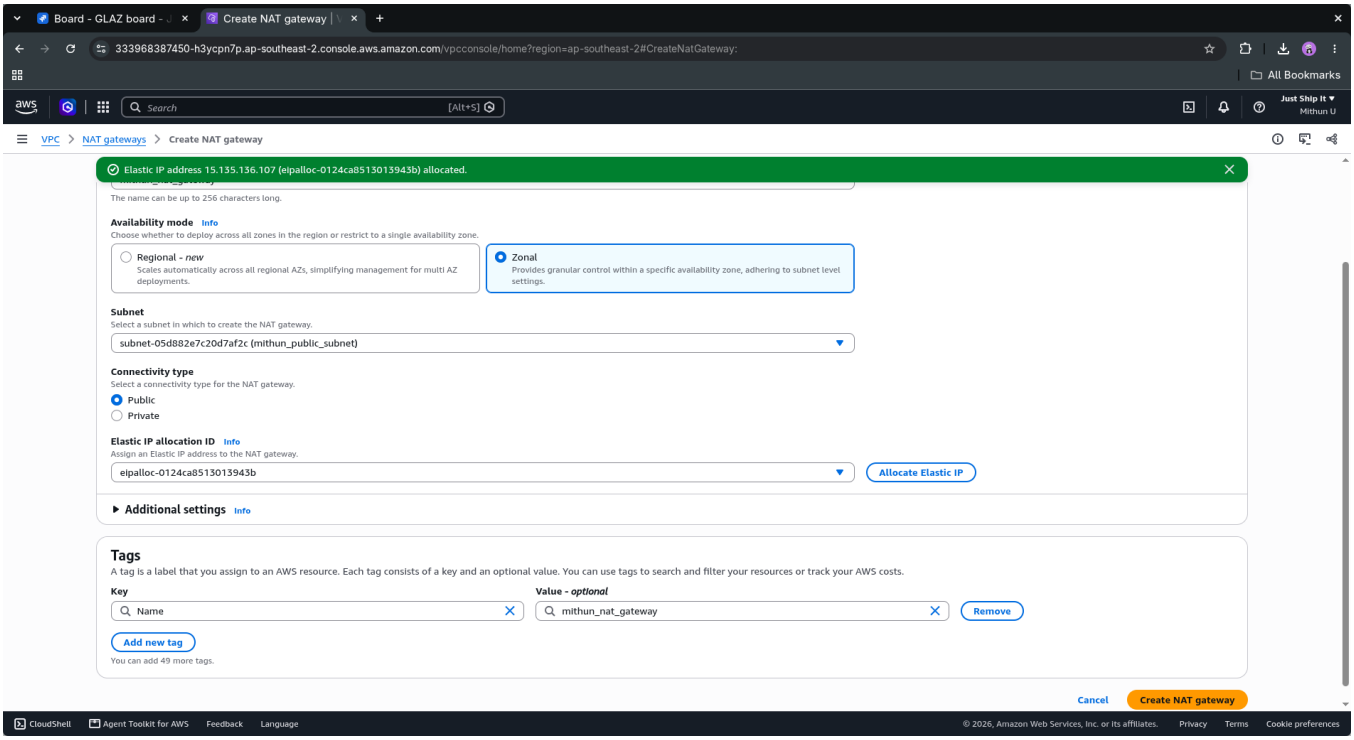



Figure 8. NAT Gateway creation settings.

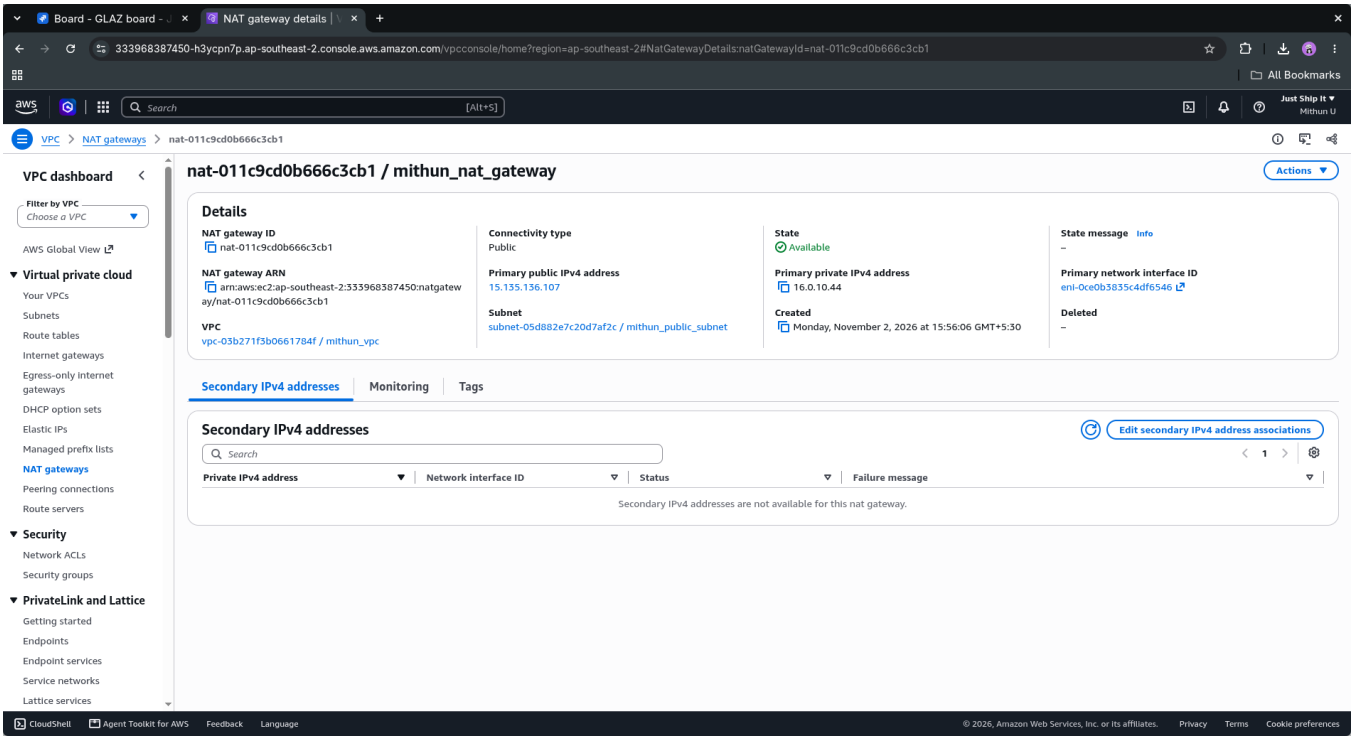



Figure 9. NAT Gateway available with an Elastic IP.

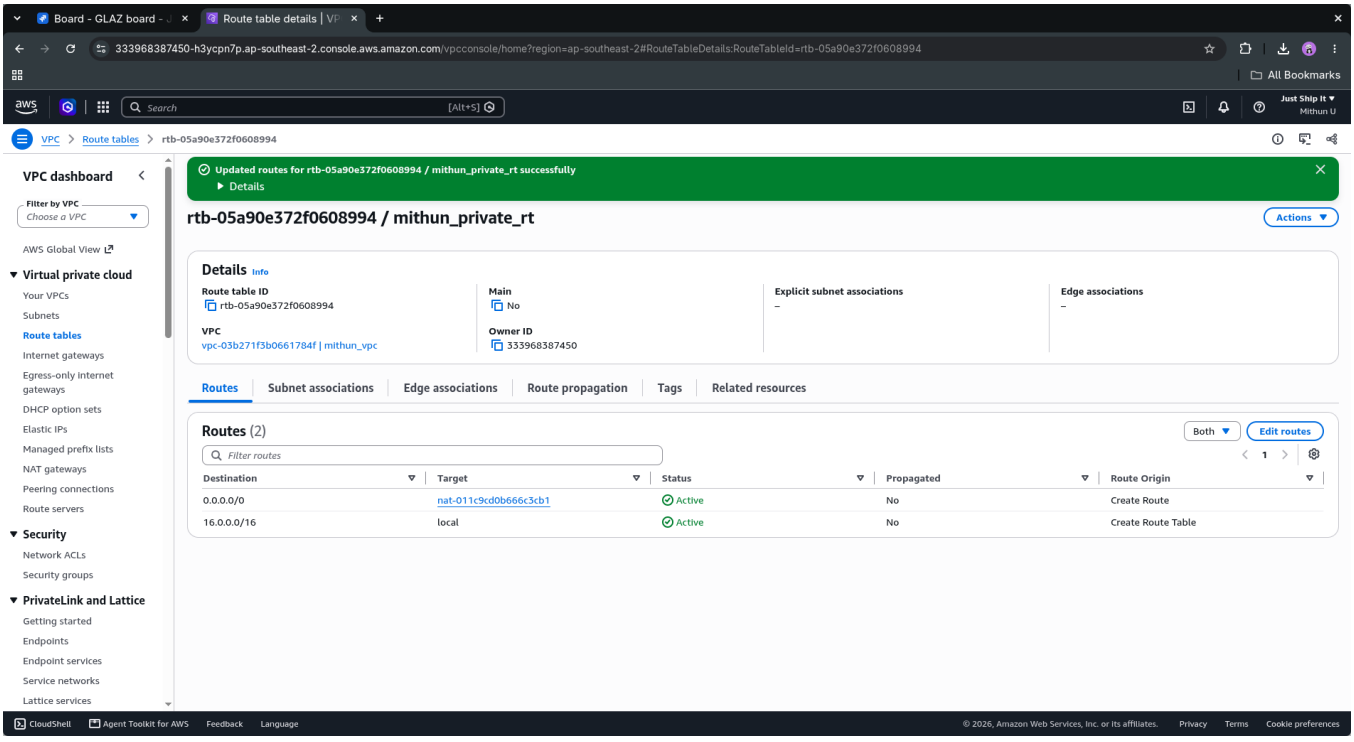



Figure 10. Private route table routes.

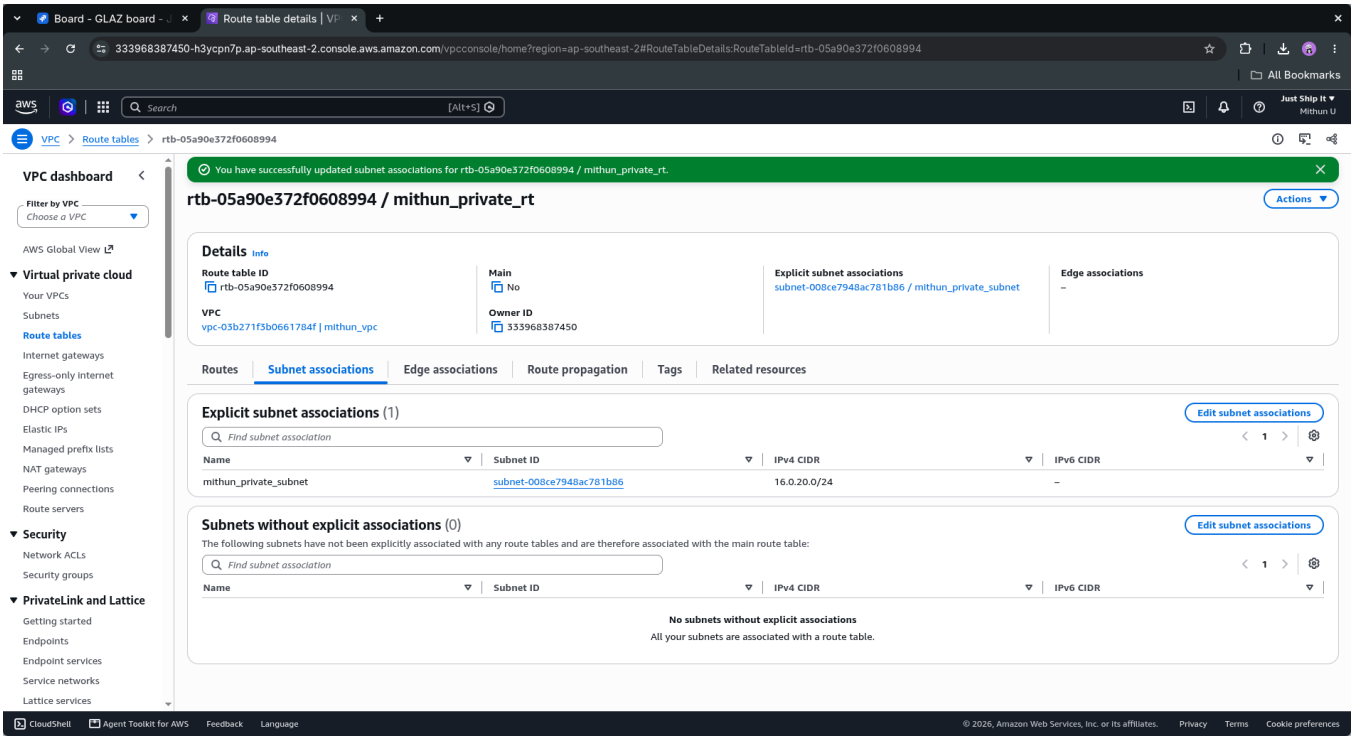



Figure 11. Private subnet association.

# 5. Configure security groups

Created two security groups in mithun_vpc with the following inbound rules.

| Security group | Inbound rules |
|---|---|
| mithun_public_sg | SSH (TCP 22) from administrator IP 14.99.219.164/32<br>HTTP (TCP 80) from 0.0.0.0/0<br>HTTPS (TCP 443) from 0.0.0.0/0 |
| mithun_private_sg | All TCP ports from mithun_public_sg |

**Note:** The private security group allows all TCP ports from the public security group as instructed for this exercise. For production use, narrow this to only the required ports.

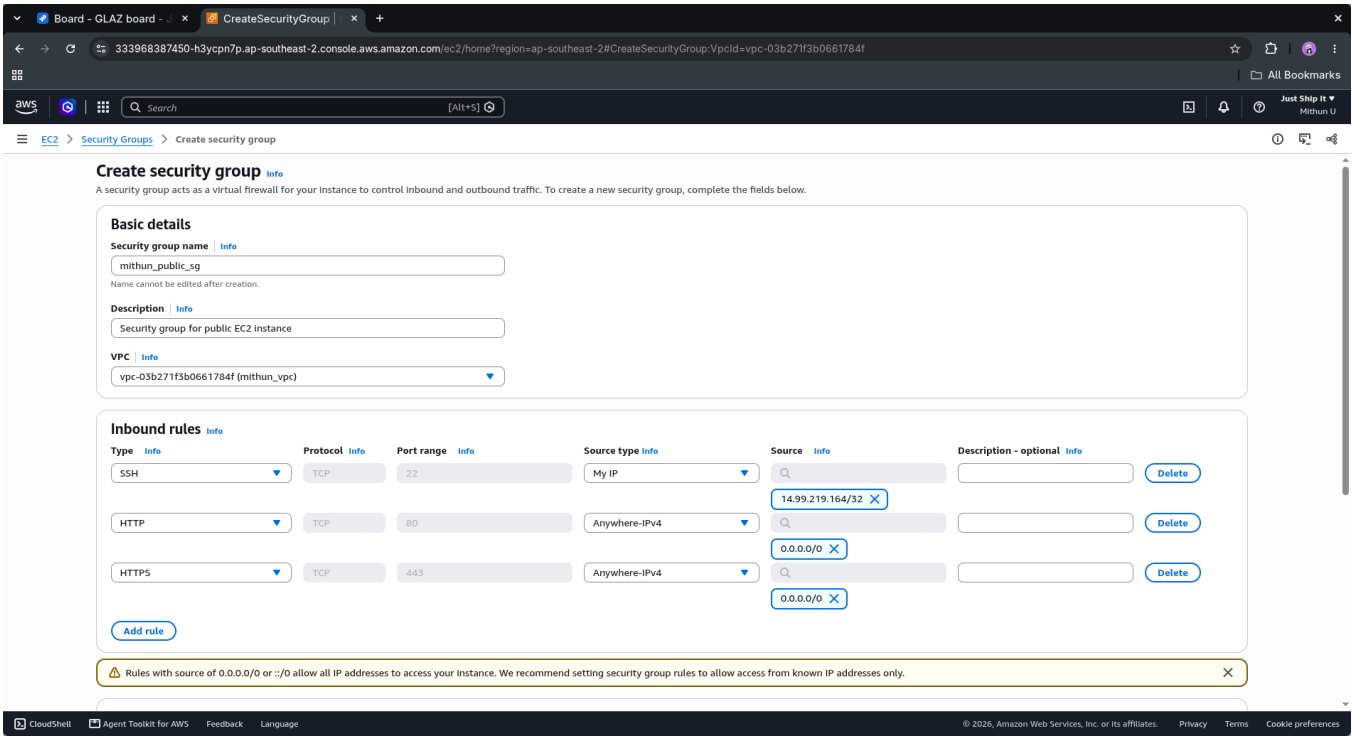



Figure 12. Public security group inbound rules.

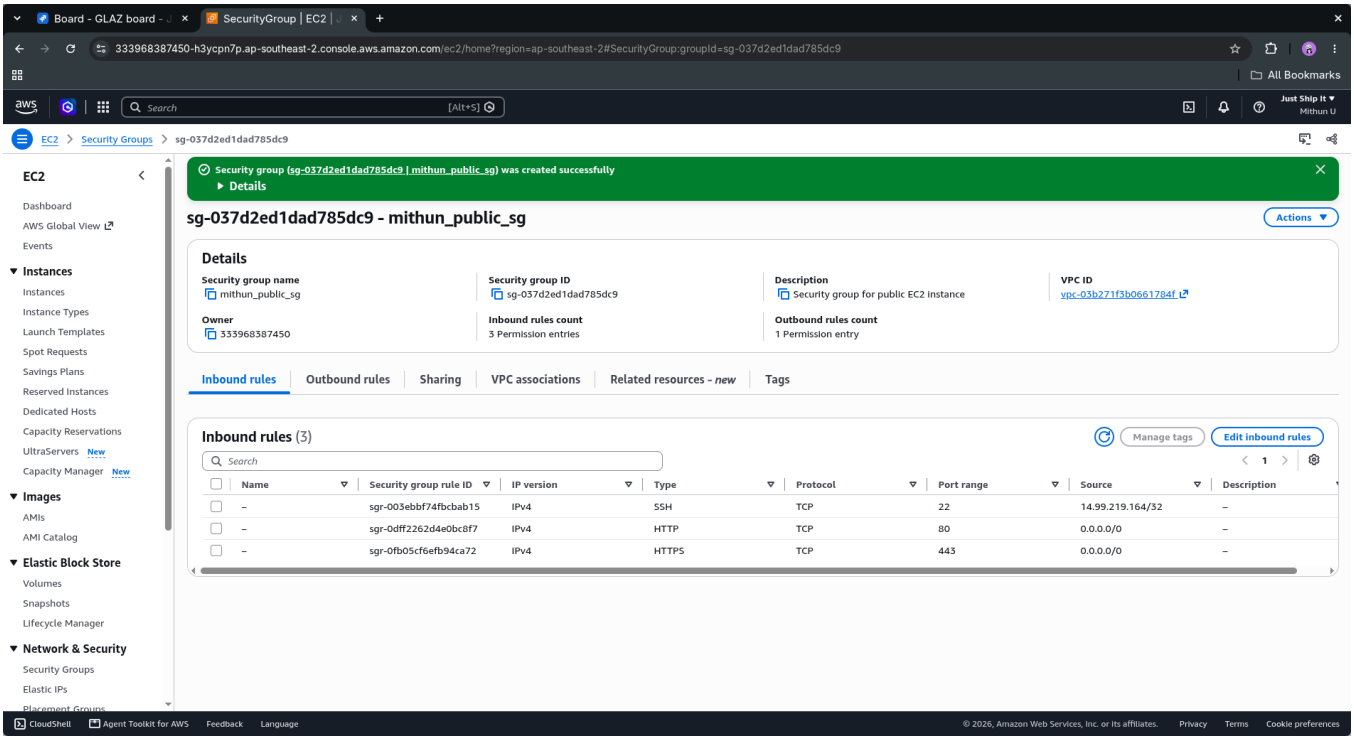



Figure 13. Public security group created.

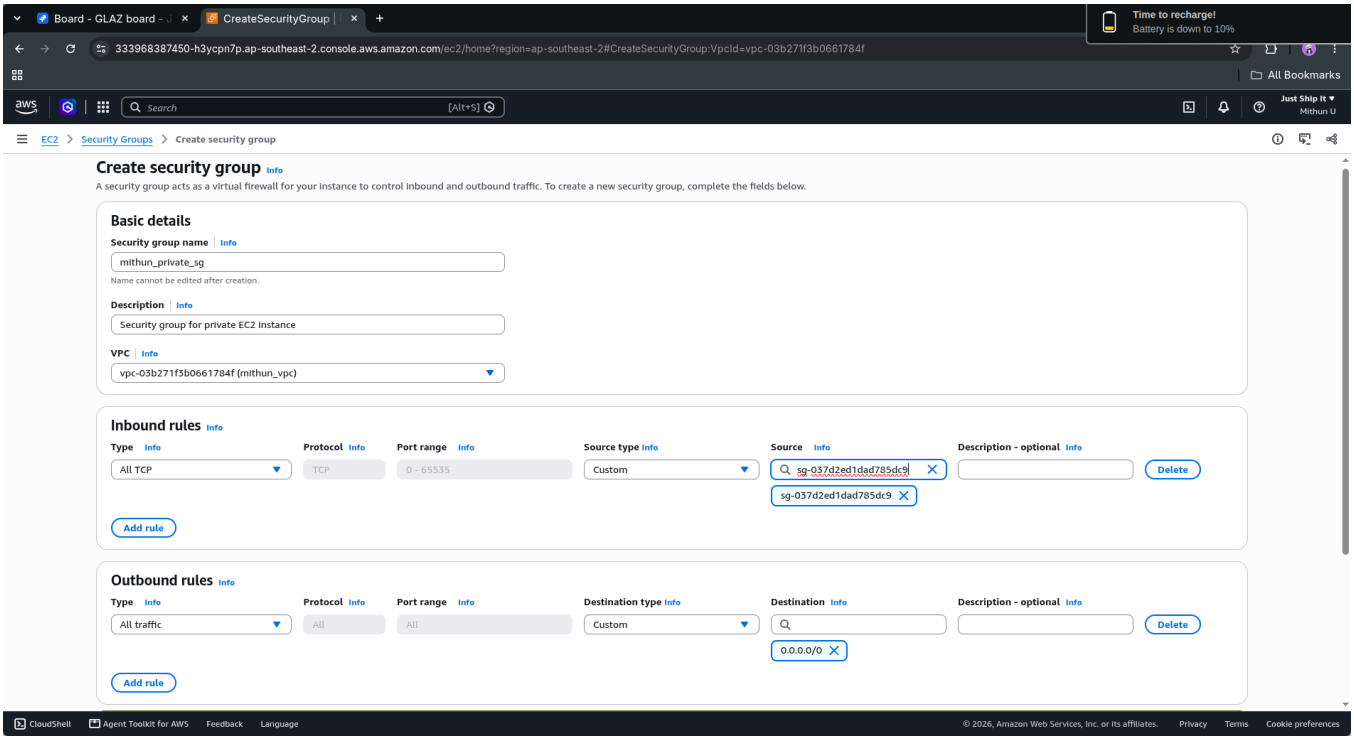



Figure 14. Private security group inbound rule.

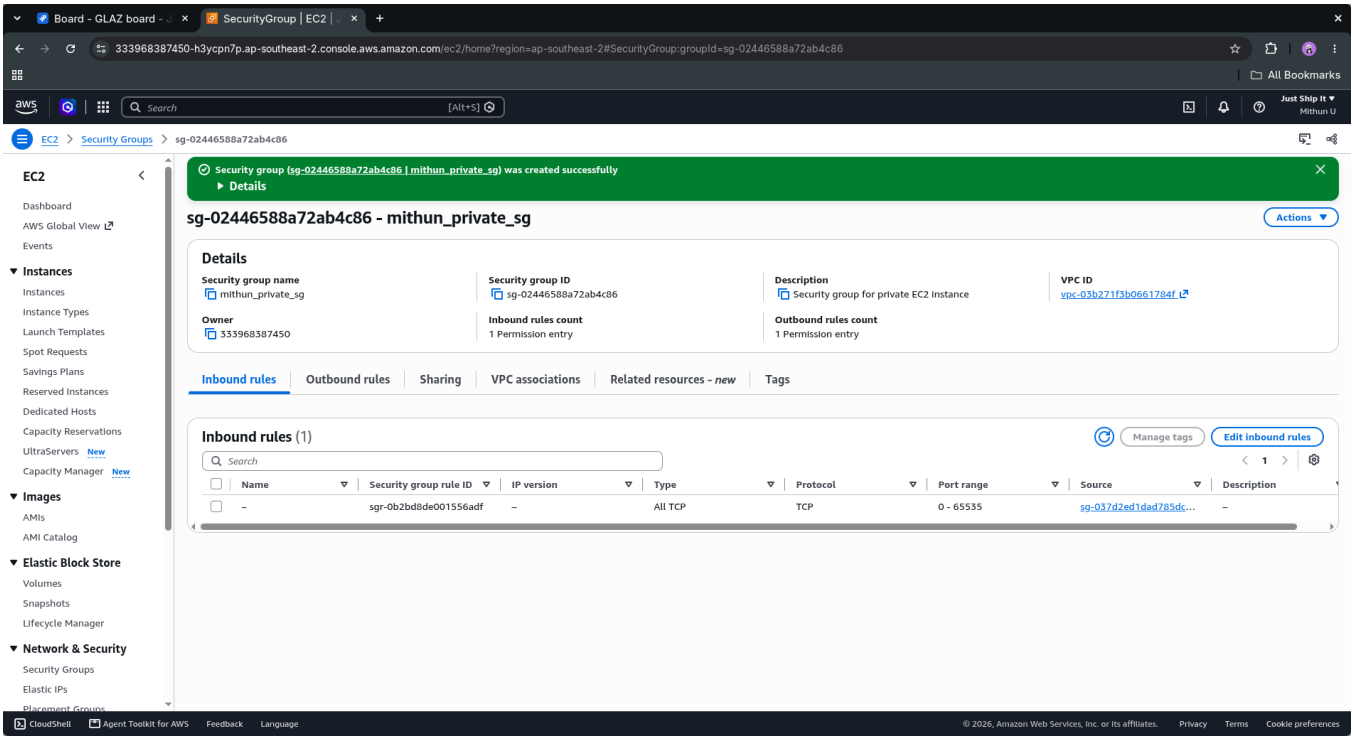



Figure 15. Private security group created.

# 6. Launch the EC2 instances

Launched two Ubuntu EC2 instances using the **t3.micro** instance type and the **kp-1** key pair. The public instance was placed in the public subnet with automatic public IP assignment enabled. The private instance was placed in the private subnet with public IP assignment disabled.

| Setting | Public EC2 | Private EC2 |
|---|---|---|
| Name | mithun_public_ec2 | mithun_private_ec2 |
| Subnet | mithun_public_subnet | mithun_private_subnet |
| Private IPv4 | 16.0.10.93 | 16.0.20.190 |

| Setting | Public EC2 | Private EC2 |
|---|---|---|
| Public IPv4 | 3.25.115.142 | None |
| Security group | mithun_public_sg | mithun_private_sg |

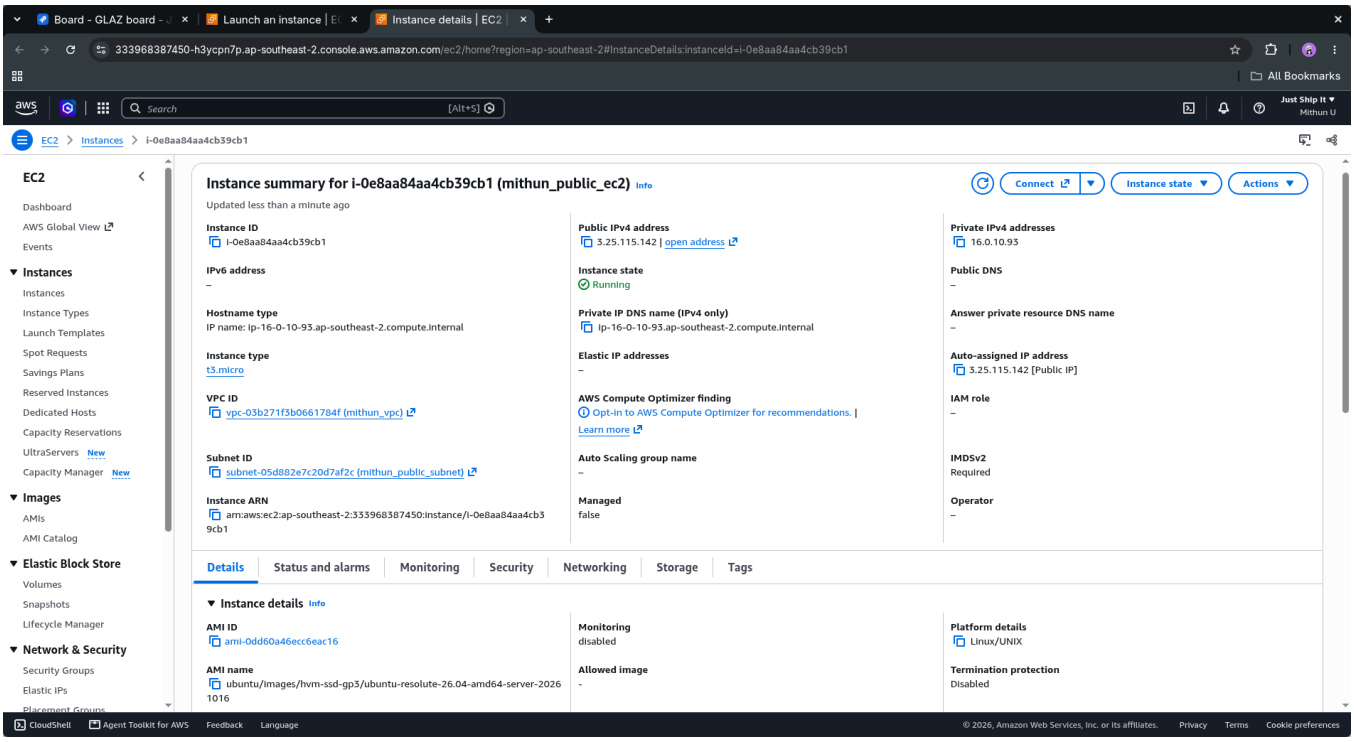



Figure 16. Public EC2 instance running with a public IPv4 address.

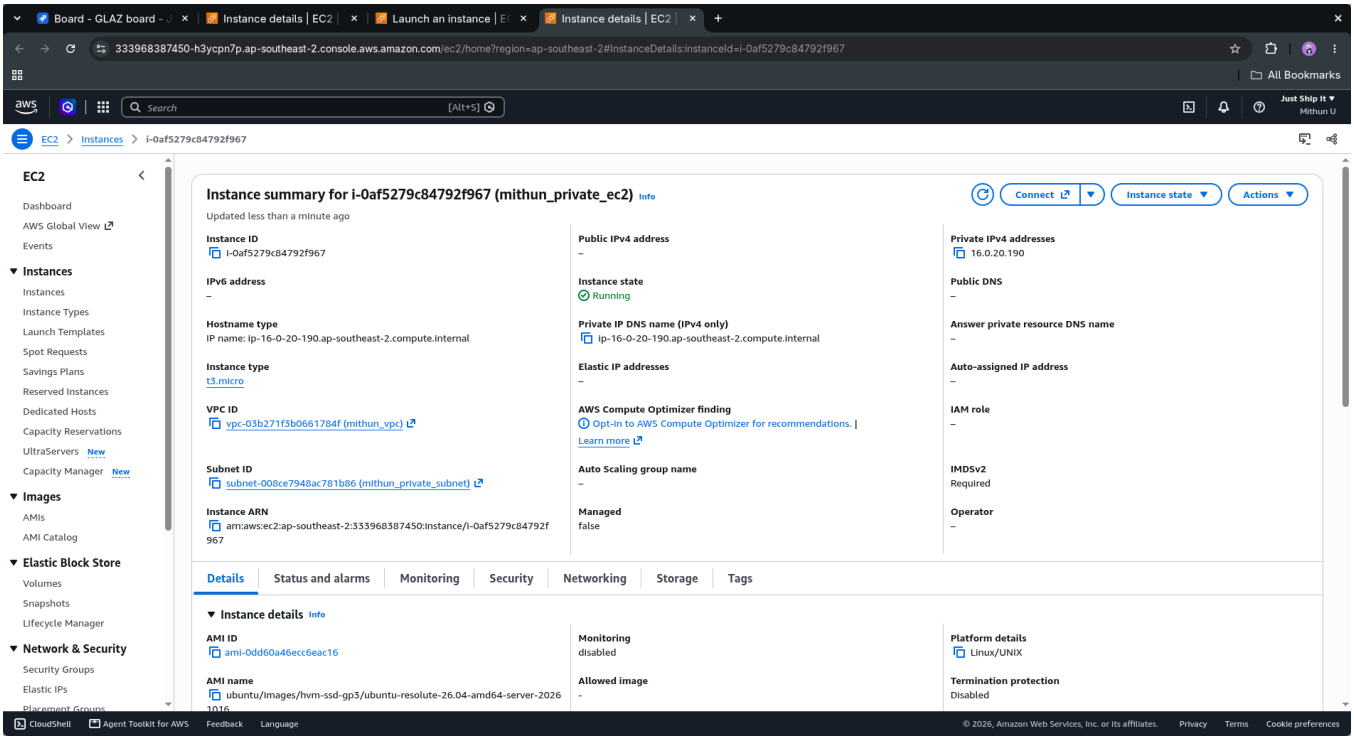



Figure 17. Private EC2 instance running without a public IPv4 address.

# 7. Connect to the private EC2 through the public EC2

First connected to the public EC2 instance from the local machine using SSH. Then, from the public EC2 instance, opened an SSH connection to the private EC2 instance using its private IPv4 address.

```
ssh -i "kp-1.pem" ubuntu@3.25.115.142

ssh -i "kp-1.pem" ubuntu@16.0.20.190
```

The second SSH connection succeeded. This confirms that the private EC2 instance can be reached through the public EC2 instance over the VPC's internal network, even though the private instance has no public IPv4 address.




Figure 18. SSH connection to the public EC2 instance.




Figure 19. SSH connection initiated to the private EC2 instance.




Figure 20. Successful SSH login to the private EC2 instance.

# 8. Host the login page on the private EC2

Installed Nginx on the private EC2 instance, enabled the service, and created a custom static login page at **/var/www/html/index.html**.

```
sudo apt update
sudo apt install nginx -y
sudo nano /var/www/html/index.html
sudo systemctl enable --now nginx
```

Verified that Nginx served the page locally from the private EC2 instance:

```
curl http://localhost
```

The response contained the custom HTML for the Private Login page, including username and password fields and a Login button. This confirms that the private EC2 instance is serving the requested page through Nginx.



Figure 21. Nginx enabled and custom private login page returned by curl.